

TECHNICAL DEEP DIVE

多模态强化学习工程解析：VeRL-Omni 架构与 MiniMax-H3 训练实录

从底层数据流到显存优化的全链路拆解

基于技术演讲与配套材料整理

配套材料：Minimax-RL-verl-omni-slides.pdf

2026-09-30

原视频：[基于 VeRL-Omni 的 MiniMax-H3 RL 后训练](#) · 配套资料：[VeRL-Omni X MiniMax-H3 RL 课件](#)

文本大模型的强化学习训练已形成成熟流水线，但当同一套框架面对图像、视频甚至音视频联合生成时，底层的生成范式差异会引发一连串工程失配 - 采样后端不兼容、显存调度失效、训练更新逻辑无法复用。本文以开源框架 VeRL-Omni 和 33B 参数的 MiniMax-H3 模型为主线，从范式差异出发，沿架构设计、算法选型、工程排雷、显存切分、吞吐优化逐层展开，完整拆解多模态扩散模型后训练的关键决策与落地细节。

适读人群：AI 系统工程师、大模型训练研究员、多模态算法工程师。

前置知识：

- 强化学习基础（PPO / GRPO 的策略梯度流程）
- 扩散模型原理（去噪迭代与流匹配）
- 分布式训练基础（张量并行 / LoRA 微调）

阅读目标：

- 1 理解文本 RL 与扩散 RL 在底层数据流上的结构性差异
- 2 掌握 VeRL-Omni 的三引擎解耦架构与异步通信机制
- 3 识别多模态模型接入通用框架时的时间步、权重同步和 Token 漂移三类隐性故障
- 4 学习大参数编码器在有限显存下的并行切分策略与访存受限场景的吞吐优化方法

一、范式冲突：文本生成与多模态扩散的强化学习差异

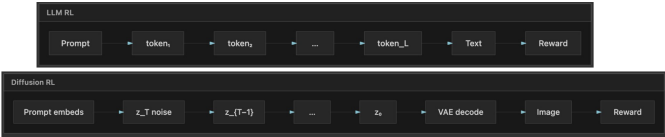
同一套训练栈为何失灵

在文本大模型领域，基于 PPO/GRPO 的强化学习已有标准流程：模型以自回归（Autoregressive, AR）方式逐 token 解码，采样完成后对整条序列做一次策略梯度更新。然而当工程团队尝试将同一框架搬到图像或视频生成任务时，立即遭遇底层生成范式的根本性冲突 - 扩散模型（Diffusion Model，通过多步去噪将随机噪声逐步还原为目标数据的生成模型）不产出离散 token 序列，而是在连续潜空间中执行多步去噪，其采样轨迹的形态、长度特征以及梯度回传路径与 AR 解码截然不同。

下图以多个维度并列展示两种范式在 RL 训练中的差异，下方给出两条生成管线的流程示意，是贯穿全文所有工程决策的出发点。

LLM RL vs Diffusion RL

	LLM RL	Diffusion RL
Use cases	Text generation, reasoning, coding	Instruction-following multimodal generation
Rollout Sampling	AR-token decoding, kv-cache L tokens, $\log P$	Multi-step diffusion on latent space (+SDE stochastic) Img/Vid, T-step trajectory (timesteps, latents, emb, $\log P$)
Sequence length	Varied — long-tail decoding bubbles	Fixed (mostly)
Rollout Backend	vLLM	vLLM-Omni
Algorithm (Advantage)	PPO, GRPO, DPO	FlowGRPO, Diffusion DPO, DiffusionNFT
Reward	Rules, text scorer (low payload)	Preference models, visual/audio VLM (high payload)
Actor (Train)	fixed token sequence $\rightarrow$ per-token $\log P \rightarrow$ single-sequence PG update (one pass)	fixed latent trajectory $\rightarrow$ per-step $\log P \rightarrow$ multi-step PG update (<i>TSDE</i> forwards/backwards)



VeRL-Omni | vLLM-Omni

图注：LLM RL 与 Diffusion RL 的对比总览，含下方两条生成管线示意。来源：演讲 PPT，第 3 页

图中上半部分为多行对比表，下半部分为两条生成管线：

- LLM RL 管线（上方）：Prompt $\rightarrow$ 逐 token 解码 $\rightarrow$ 文本 $\rightarrow$ 奖励
- Diffusion RL 管线（下方）：Prompt 嵌入 $\rightarrow$ 噪声 $\rightarrow$ 多步去噪得到潜变量 $\rightarrow$ VAE 解码 $\rightarrow$ 图像 $\rightarrow$ 奖励

三个最关键的差异维度如下。

差异一：采样方式与轨迹结构

维度	LLM RL	Diffusion RL
采样过程	AR token 解码，依赖 KV-Cache 加速	潜空间多步去噪，可含 SDE 噪声
轨迹产物	一维 token 序列 + 每 token 对数概率	T 步时间步 $\times$ 潜变量 $\times$ 每步对数概率

AR 解码每步仅生成一个标量 token；而扩散采样的每一步都输出一个与图像分辨率相关的完整潜变量张量。rollout 阶段的显存占用与吞吐特征由此产生根本分歧：文本 RL 的瓶颈在长尾解码气泡，扩散 RL 的瓶颈在于 T 步轨迹的批量存储与传输。

差异二：序列长度 - 可变与固定

文本生成的序列长度高度可变，短回复可能几十 token，长推理链可达数千。由此产生的解码气泡（decoding bubble） - 同一 batch 内短序列等待长序列完成 - 是 vLLM 等推理引擎重点优化的对象。

扩散模型的去噪步数 T 在推理前即确定（如 20 步、50 步），同一 batch 内所有样本的轨迹长度一致。长度固定消除了解码气泡，但引入了新问题：每步去噪都是一次完整的前向传播，计算密度远高于单 token 解码。传统 vLLM 的 KV-Cache 调度与连续批处理在此场景下无用武之地。

差异三：训练更新路径

- LLM RL：冻结 token 序列，计算每 token 对数概率，执行单次策略梯度更新。
- Diffusion RL：冻结潜变量轨迹，计算每步对数概率，执行多步策略梯度更新 - 需要 T 步的前向/反向传播。

多步更新使单次训练迭代计算量随 T 线性增长，对显存调度的要求远超文本场景。

框架定位

三重差异叠加后，直接后果是：原有基于 vLLM 的 LLM RL 框架在处理扩散模型时，采样后端不匹配、显存调度失效、训练更新逻辑无法复用。这正是 VeRL-Omni（专为 diffusion 和 omni 模型设计的开源多模态 RL 训练框架）的工程起点。它将推理后端从 vLLM 替换为 vLLM-Omni（VeRL-Omni 中用于高效多模态生成的高吞吐异步 rollout 后端），并在算法层支持 FlowGRPO（一种适配扩散模型的 RL 算法）、Diffusion DPO 以及 DiffusionNFT（利用前向过程与标量最优奖励的在线扩散 RL 算法，无需构建胜负对）等专用算法。

小结：文本 RL 的框架假设 - 可变长离散序列、KV-Cache 调度、单步策略梯度 - 在面对多步固定长度的连续潜变量轨迹时全面失效。VeRL-Omni 的核心定位正是从系统架构层面弥合这一范式鸿沟。下一步需要展示它是如何在架构层面协调推理、打分与训练三类异构负载的。

二、架构解构：VeRL-Omni 的异步多模态数据流

三类异构负载如何共存于一条流水线

多模态 RL 的单次迭代包含三类计算：轨迹生成需要高吞吐推理、奖励打分可能调用外部视觉语言模型或规则引擎、策略优化则是标准的梯度反传。三者对显存、算力和通信带宽的需求各不相同。若以同步串行方式执行，任何一个阶段的延迟都会阻塞整条链路。

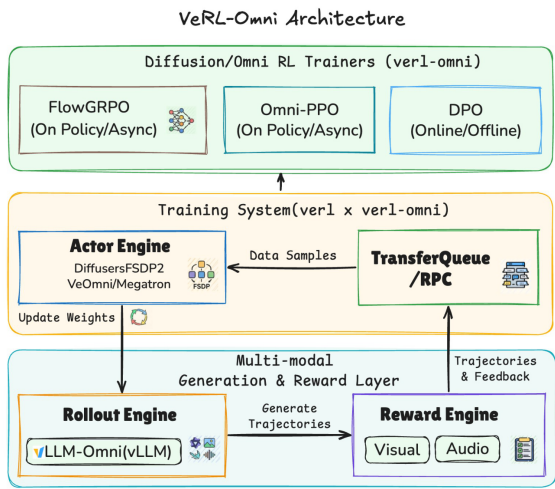
VeRL-Omni 的应对策略是将三类负载映射为三个可独立调度的引擎，再用异步队列串联，使下游阶段在上游尚未全部完成时即可启动。

三引擎 + 异步通道

下图展示了 VeRL-Omni 的系统架构总览，需要关注三个引擎之间的数据流向以及连接它们的异步通道。

VeRL-Omni

VeRL-Omni Overview



Key Features

- **Efficient multimodal rollout:** vLLM-Omni backend for high-throughput async generation (DiT, unified, omni pipelines)
- **Modular training backends:** selectable FSDP2 and VeOmni engines with combinable parallelism (DP / SP / TP)
- **Flexible reward serving:** Support for multi-reward serving (HPSv3, GenRM-OCR, UnifiedReward, etc.), HTTP scorer, and **async reward** computation to overlap the rollout phase
- **Stable & Fast diffusion trainers:** rollout correction and deterministic RL; ~20% higher E2E throughput vs diffusers-based FlowGRPO on Qwen-Image
- **Hardware support:** NVIDIA GPU and Ascend NPU

VeRL-Omni | vLLM-Omni

图注：VeRL-Omni 架构概览，呈现训练系统三大引擎及其连接方式。来源：演讲 PPT，第 4 页

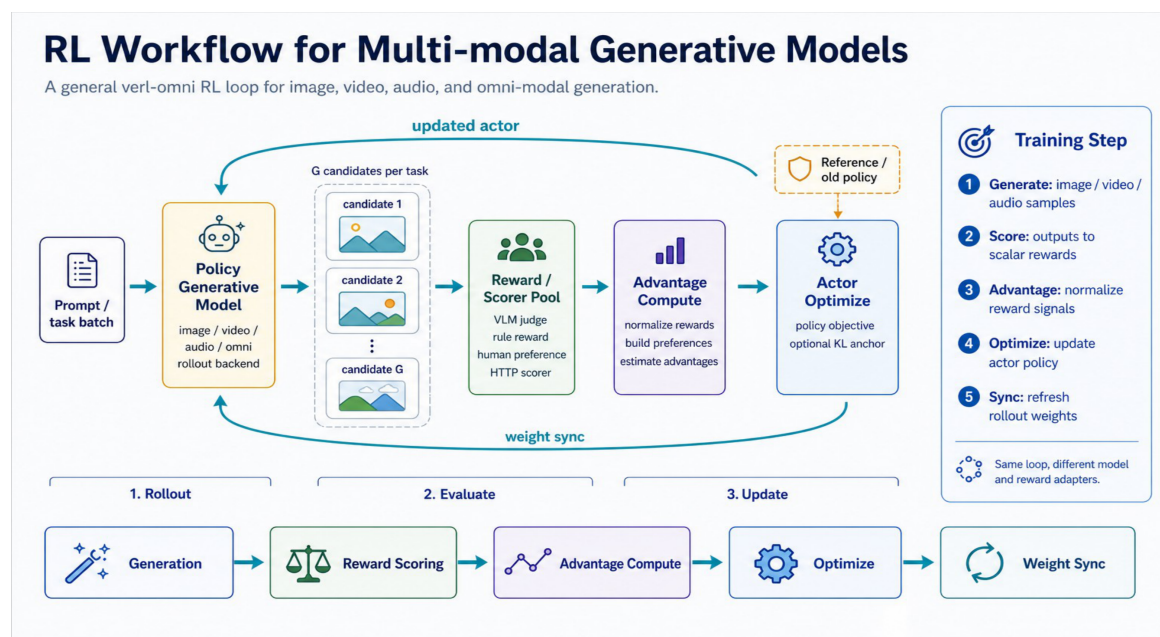
图中自上而下分为算法层与训练系统层，关键组件如下：

层级	组件	职责
算法调度	Diffusion/Omni RL Trainers	可选 FlowGRPO、Omni-PPO、DPO 等算法
Actor Engine	Diffusers / FSDP2 (PyTorch 全分片数据并行策略)	策略参数存储与梯度更新，支持 DP/SP/TP 组合并行
Rollout Engine	vLLM-Omni	高吞吐异步生成
Reward Engine	视觉 / 音频评分器	多维度打分
数据通道	TransferQueue（传输队列）/ RPC	跨引擎异步数据搬运

三条核心箭头代表数据的单向流转：1) 权重同步 (Actor Rollout)：每轮优化后将最新策略权重推送至 vLLM-Omni；2) 轨迹下发 (Rollout Reward)：候选样本送入奖励引擎打分；3) 数据回传 (Reward Actor)：打分后的轨迹与奖励信号经 TransferQueue 回送，构成训练数据。

单 Batch 生命周期：五步闭环

下图将一次迭代拆解为五个有序步骤，底部时间线归纳为三大阶段，是理解数据流转时序的关键。



VeRL-Omni | vLLM-Omni

11

图注：多模态 RL 训练的完整 workflow，展示从 Prompt 输入到权重同步的五步循环及三阶段时间线。来源：演讲 PPT，第

11 页

阶段一 · Rollout (生成)

- 步骤 1 - Generate：一批 Prompt 进入策略模型，vLLM-Omni 为每个任务产出 G 个候选样本。

阶段二 · Evaluate (评估)

- 步骤 2 - Score：G 个候选进入 Reward/Scorer Pool (奖励评分池)，支持 VLM 裁判、规则奖励、人类偏好信号及通用 HTTP Scorer 并行工作。关键设计：异步奖励计算 - 打分与下一轮 rollout 在时间上重叠，避免 GPU 空等。
- 步骤 3 - Advantage：对奖励做归一化、构建偏好对或估算优势函数，为策略梯度提供信号。

阶段三 · Update (更新)

- 步骤 4 - Optimize：Actor Engine 执行策略目标函数计算，可选 KL 锚定约束策略偏移幅度。
- 步骤 5 - Sync：更新后的权重同步回 Rollout Engine，循环进入下一迭代。

TransferQueue 的异步流水线机制

TransferQueue 是连接 Reward Engine 与 Actor Engine 的异步数据通道。当奖励引擎完成部分样本打分后，无需等待整批结果，即可将已完成数据推入队列；Actor Engine 侧从队列拉取可用数据开始预处理。这一机制将串行的“生成 → 打分 → 回传”链路改造为流水线执行，是提升端到端吞吐的关键调度手段。

吞吐量锚点

在 Qwen-Image（VeRL-Omni 支持的一种扩散生成模型）上，VeRL-Omni 相比基于 Diffusers 的 FlowGRPO 实现了约 20% 的端到端吞吐量提升（PPT 第 4 页书面标注）。提升来自两方面：vLLM-Omni 替代原生 Diffusers rollout 推理带来的生成加速，以及异步奖励计算与 rollout 的时间重叠。需注意，演讲材料未给出延迟拆分细节或不同模型规模下的量化对比，该数字仅适用于 Qwen-Image + FlowGRPO 这一特定对比条件。

小结：VeRL-Omni 通过三引擎解耦与 TransferQueue 异步流水线，将轨迹生成、奖励打分和策略优化三类异构负载并行化。在 Qwen-Image 场景下验证了约 20% 吞吐增益。掌握了通用架构之后，接下来将其落地到一个真正复杂的多模态模型上 - MiniMax-H3 - 以检验框架对极端条件的承载力。

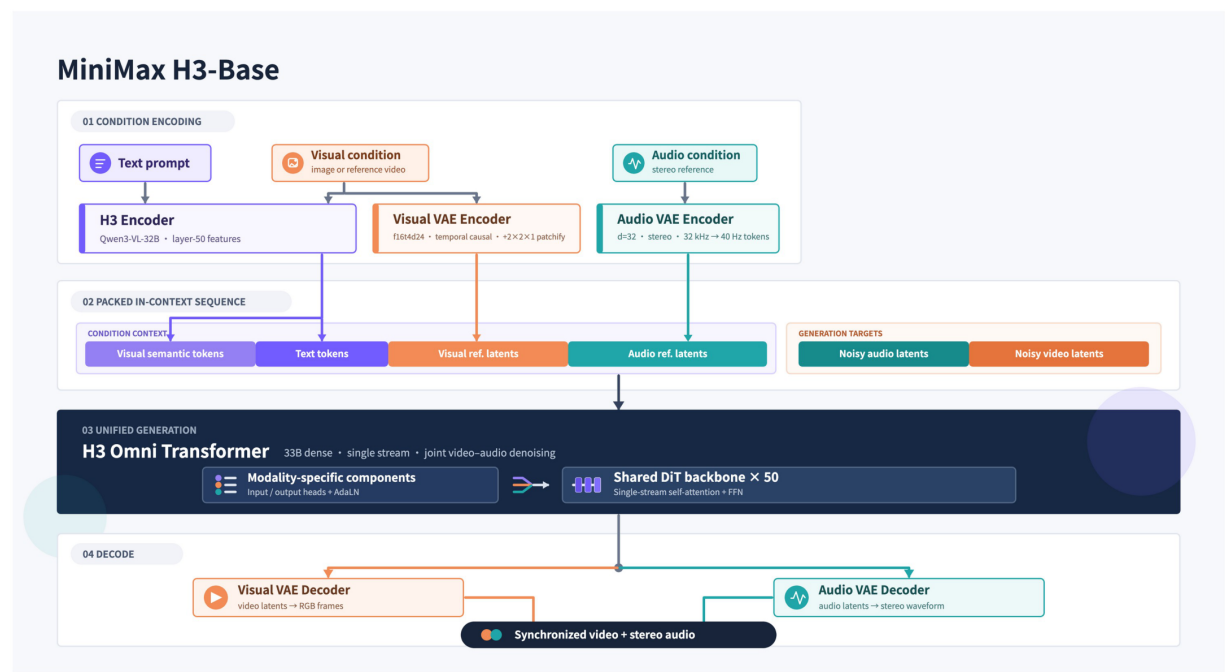
三、目标模型：MiniMax-H3 的统一生成机制与挑战

三种模态共享一条骨干带来的 RL 难题

VeRL-Omni 的通用架构是否足以应对真正复杂的多模态模型？MiniMax-H3 提供了一个极端检验场景：它同时接收文本、视觉、语音三种条件输入，并用同一组参数生成同步的音视频。当 RL 训练面对如此庞大且耦合的状态空间时，显存预算、计算开销和多模态奖励对齐都构成独立挑战。

架构全景

下图展示 MiniMax-H3 从条件编码到统一生成再到音视频解码的完整数据流，理解该流程是后续所有工程决策的基础。



图注：MiniMax-H3 架构总览，展示从三种条件编码到共享 DiT 骨干再到音视频解码的完整流程。来源：演讲 PPT，第 8 页

图中自左向右分为四个阶段：条件编码（Condition Encoding）→ 打包序列（Packed In-Context Sequence）→ 统一生成（Unified Generation）→ 解码输出（Decode）。

条件编码：三路输入如何汇聚

条件编码模块负责将异构的文本、视觉和语音信号转换为统一的潜空间表示，以便送入共享骨干网络。

模态	编码路径	输出形式
文本	经一个 32B 参数的语言模型提取深层隐藏特征	文本条件向量
视觉	双分支 - 一路经同一语言模型提取语义特征，另一路经视觉 VAE（Variational Autoencoder，变分自编码器，将像素映射到低维连续潜空间的编码器）获得视觉潜码	视觉条件 + 视觉潜码
语音	经 Audio VAE，将音频信号转换为离散潜码	语音潜码

三路结果被拼装为一条 Packed Sequence，以专用 token ID 区分模态边界，再通过因果注意力驱动自回归生成。仅条件编码阶段就涉及一个 32B 级别的编码器和两个 VAE，显存占用在编码侧已十分可观。

统一生成：共享 DiT 骨干

图中核心方块标注为 H3 Omni Transformer，即 DiT（Diffusion Transformer，将扩散过程与 Transformer 注意力结合的生成架构）骨干网络，共 50 层、33B dense 参数。所有模态的潜变量在同一骨干内通过共享注意力权重与 AdaLN（Adaptive Layer Normalization，自适应层归一化）调制参数进行交互。统一生成模块的输出被拆分为视觉和语音两条通道，分别送入对应的 VAE Decoder 还原为像素与波形。

对 RL 训练的三层压力

- 1 **显存压力**：33B dense 模型在全精度下单卡无法容纳；RL 训练额外需要保存参考策略权重与优化器状态。32B 编码器同样需要常驻显存或按需卸载（第六节详述解法）。
- 2 **计算开销**：50 层 Transformer 的前向-反向传播叠加多步去噪迭代，单次 rollout 计算量远超纯文本自回归。
- 3 **多模态奖励对齐**：视频质量、音画同步、语义一致性分别需要不同评价指标。将它们压缩为单一标量奖励时，各模态间的梯度方向可能冲突。

考虑最简退化情形 - 仅输入文本提示、无参考视频、无语音条件 - 视觉和语音编码路径仍需初始化（空 token 填充），33B 骨干的完整前向计算不可省略。即便输入退化，计算量几乎不变。

小结：MiniMax-H3 将三种模态统一压入 33B / 50 层的共享 DiT 骨干，在推理端实现了音视频同步生成，但也将显存、多步扩散计算和模态间奖励冲突一并带入 RL 训练环节。面对如此规模的模型，传统依赖构建胜负对的偏好对齐方法在采样效率上难以直接适用，需要一种绕过成对数据的算法路径。

四、算法映射：DiffusionNFT 与标量最优奖励

成对偏好数据在连续生成空间中的代价

在大语言模型的偏好对齐中，构造“胜者 / 败者”回答对相对直接。但切换到图像或视频等连续动作空间后，同一提示词可生成近乎无穷的像素组合，逐对评判成本高、噪声大，且成对构造的数量随候选数呈组合增长。

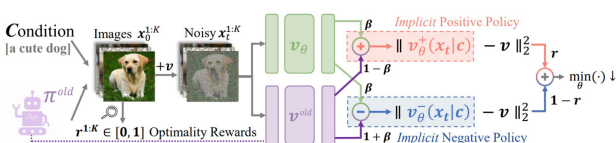
DiffusionNFT 提供了另一条路径：用一个 $r \in [0,1]$ 的标量最优奖励（Scalar Optimality Reward）直接加权每个样本，将偏好信号从“两个样本之间的比较”压缩为“单个样本的质量打分”。MiniMax-H3 的多模态 RL 训练正是基于该算法接入 VeRL-Omni。

机制全景

下图展示了 DiffusionNFT 的奖励加权流匹配原理，包括隐式正负策略的构造方式和训练曲线，是理解该算法如何绕过成对数据的关键。

RECIPE 02 · REWARD-WEIGHTED FLOW MATCHING

MiniMax-H3 × DiffusionNFT



- NO PAIRS** A scalar optimality reward $r \in [0, 1]$ weights each sample — no winner/loser construction.
- IMPLICIT ±** Positive / negative policies are interpolations of one network with its frozen `old` adapter.



KEY CONFIG

```
..model.algorithm=diffusion_nft
..model.model_type=diffusion_nft_model
algorithm.sample_source=online
algorithm.paired_preference=false
..diffusion_loss.loss_mode=diffusion_nft
..model.policy_state_adapters=['default','old']
```

IMPLEMENTATION — DIFFUSION_ALGOS.PY

```
@register_diffusion_loss("diffusion_nft")
def compute_diffusion_loss_nft(
    old_log_prob, log_prob, advantages, config):
    beta = config.diffusion_loss.nft_beta
    # implicit +/- policies via frozen old adapter
    v_pos = beta*v_theta + (1-beta)*v_old
    v_neg = (1+beta)*v_old - beta*v_theta
    # r in [0,1] weights positive vs negative branch
    loss = (r * mse(v_pos, v_target)
            + (1-r) * mse(v_neg, v_target)).mean()
    return loss, {"actor/nft_loss": loss.item()}
```

DiffusionNFT: Online Diffusion Reinforcement with Forward Process, 2026 ICLR Oral

VeRL-Omni | vLLM-Omni

图注：MiniMax-H3 × DiffusionNFT 的算法架构、关键配置与训练曲线。来源：演讲 PPT，第 12 页

图中从左到右包含三块信息：左侧为条件输入到候选采样的流程，中部为隐式正策略与隐式负策略的插值公式，右侧为实际训练代码片段与配置开关。下方曲线展示训练过程中平均奖励的上升趋势。

隐式正负策略：一个网络，两个角色

DiffusionNFT 不训练两个独立的策略网络，而是将当前策略 v_{θ} 与冻结的旧适配器 v_{old} 做线性插值，隐式构造出正策略和负策略：

$$v_{pos} = \beta v_{\theta} + (1 - \beta) v_{old}$$

$$v_{neg} = (1 + \beta) v_{old} - \beta v_{\theta}$$

其中 β （代码中 `config.diffusion_loss.nft_beta`）控制新旧策略的混合程度。 β 越大，正策略越偏向当前网络，负策略越远离当前网络。 v_{old} 在每轮更新开始时冻结，充当锚点防止策略漂移过快。

配置中 `policy_state_adapters=['default', 'old']` 对应两个角色 - `default` 为可训练的当前适配器，`old` 为冻结副本。

标量奖励驱动损失函数

最终损失将正负两条分支用标量奖励 r 加权：

$$L = r \cdot \text{MSE}(v_{pos}, v_{target}) + (1 - r) \cdot \text{MSE}(v_{neg}, v_{target})$$

r 越高，正策略分支权重越大，网络被推向“生成更好样本”的方向； r 越低，负策略分支主导，网络被推离“生成差样本”的方向。

最小例子：极端值下的行为

当 $r = 1$ (满分样本) : $L = \text{MSE}(v_{\text{pos}}, v_{\text{target}})$ 。仅优化正策略分支，网络朝当前适配器方向强化，等价于对该优质样本做标准流匹配 (Flow Matching，通过学习连续速度场将噪声分布映射到数据分布的生成方法) 回归。

当 $r = 0$ (零分样本) : $L = \text{MSE}(v_{\text{neg}}, v_{\text{target}})$ 。仅优化负策略分支。由于 v_{neg} 中 v_{θ} 前系数为 $-\beta$ ，梯度方向实际使 v_{θ} 远离该样本对应的速度场，起到"排斥"作用。

当 $0 < r < 1$: 两个分支按比例混合，形成连续的"吸引 / 排斥"梯度场 - 无需离散化为二元标签，奖励信号被完整保留。

为何优先接入 DiffusionNFT

据 VeRL-Omni 工程团队在技术分享中的说明，DiffusionNFT 相较 FlowGRPO 省去了 ODE 到 SDE 的转换流程，直接在干净图像上反向加噪并学习该过程，接入复杂度更低。这也是 MiniMax-H3 训练中首先落地的算法选择。

边界： 演讲材料未给出 β 的推荐取值范围或消融实验数据。配置项 `paired_preference=false` 是启用该算法的必要开关，若误设为 `true` 将回退到成对偏好逻辑。此外，整个机制依赖 r 的准确性 - 失去了相对比较的校准，对奖励模型绝对值的精度要求更高。理论算法确定后，实际工程落地会遭遇框架与模型之间多处隐性约定冲突，需要逐一排查。

五、工程排雷：时间步对齐与 Token 漂移修复

三处隐性约定冲突

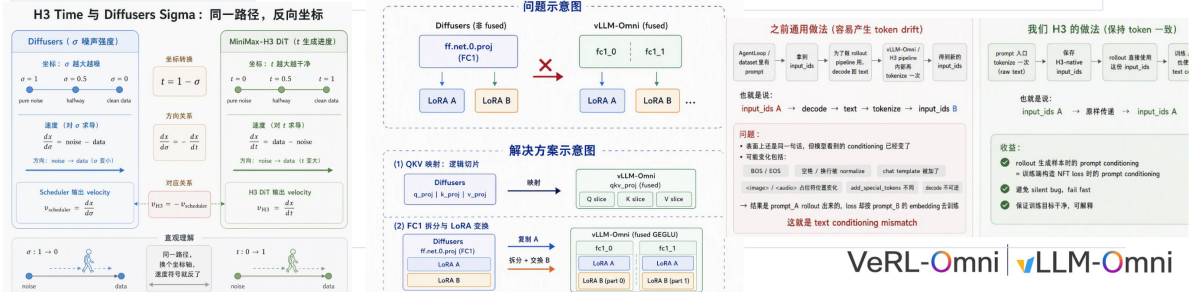
将 MiniMax-H3 接入 VeRL-Omni 的过程中，一个典型现象是 - loss 曲线看似在下降，但生成质量不升反降，甚至输出完全无语义的噪声。根本原因并非算法逻辑有误，而是模型与框架之间存在三处隐性约定冲突。它们不会在编译或启动阶段报错，只在运行时以"不收敛"或"语义丢失"的形式静默显现。

下图集中展示了三类工程故障的根因与修复策略，是本节分析的入口。

Fix pattern after those pitfalls

Turn each runtime failure into an explicit contract before presenting results

Pitfall	Fix used	Solve
Timestep convention mismatch	Convert scheduler timestep to H3 DiT timestep with $t_{H3} = 1 - \sigma$; flip H3 velocity before using generic flow-matching loss	H3 uses data-progress time, while the generic scheduler uses noise-level time
LoRA mismatch	Use explicit QKV, FC1, and FC2 targets. Fail fast on all-linear	Only sync weights the rollout backend can load
Prompt token drift	Preserve H3-native <code>input_ids</code> ; avoid decode / re-tokenize	Training must use the same text conditioning as rollout



图注：MiniMax-H3 后训练中三个关键工程陷阱及其修复方式。来源：演讲 PPT，第 14 页

故障类型	根因	修复策略
时间步约定不匹配	H3 采用数据进度时间，通用调度器采用噪声级别时间	转换 $t_{H3}=1-\sigma$ ，翻转速度场
LoRA 权重切分不一致	rollout 后端对 QKV、FC 层的打包方式与训练端不同	显式指定 QKV / FC1 / FC2 目标，仅同步可加载权重
Prompt Token 漂移	重新解码再分词引入额外控制 token	保留 H3 原生 <code>input_ids</code> ，禁止 decode/re-tokenize

5.1 时间步约定不匹配：方向相反的时间轴

问题：Flow Matching 类扩散模型通过时间变量 t 在噪声与数据之间做插值，但不同实现对于 t 的语义约定截然相反：

- 通用扩散调度器（Diffusers 等）： σ 从 1 到 0， $\sigma=1$ 为纯噪声， $\sigma=0$ 为干净数据。
- H3 DiT：使用数据进度时间（data-progress time）， $t=0$ 为噪声， $t=1$ 为完整视频。

若直接把调度器的 σ 传入 H3，模型在本该去噪的时间步上执行加噪，速度场方向完全翻转。loss 值可能仍在下降 - 因为网络有能力拟合任何方向的速度场 - 但生成内容已与目标分布背道而驰。

修复机制：VeRL-Omni 在调度器输出与 H3 前向计算之间插入算术转换： $t_{H3} = 1 - \sigma$ ，同时对 H3 输出的速度场取反，使梯度方向与通用 Flow Matching loss 保持一致。该操作不改变网络结构或权重，仅做数值映射。

边界：此修复仅针对"数据进度时间"约定。接入其他扩散模型时，必须先确认其时间轴方向，否则同一转换反而引入新错误。

5.2 LoRA 权重切分不一致：训练端与推理端的矩阵布局冲突

问题：RL 训练循环中，训练端与 rollout 后端需要频繁同步 LoRA（Low-Rank Adaptation，低秩适配微调，仅训练少量增量参数的高效微调方法）增量权重。但两端对同一层的存储方式不同：

- 训练端：Q、K、V 投影矩阵各自独立存储；FC 层为完整的单一权重矩阵。
- vLLM-Omni：为加速推理，将 Q、K、V 打包合并为一个矩阵；在张量并行下将 FC 层拆为多个分片，分片排列顺序可能与训练端相反。

当使用 `all-linear` 等通配目标自动注入 LoRA 时，两端对“同名层”的内部结构定义不一致，权重同步静默失败 - rollout 端实际使用的仍是旧权重或错位权重，策略梯度更新等效于被丢弃。

修复机制：1) LoRA 注入点严格声明为 QKV、FC1、FC2 三类显式目标，而非通配匹配。2) 同步时按目标名逐一映射训练端与 rollout 端的切分关系，包括分片顺序重排。3) 若某目标在 rollout 端无法加载，立即终止（fail fast），杜绝静默回退。

最小例子：训练端持有完整 FC1 矩阵，vLLM-Omni 将其拆分为 `shard_0` 与 `shard_1`，且排列顺序可能翻转。同步逻辑须按 rollout 端规则切分、重排后写入，LoRA 增量才能在推理时正确生效。

5.3 Prompt Token 漂移：往返编解码引入的隐性偏差

问题：Token 漂移（Prompt Token Drift）指训练时的文本条件编码与 rollout 时不一致，导致模型在两阶段“看到”的 prompt 发生语义偏移。

此前的通行做法是：从数据集拿到 `input_ids` 后，在 rollout 端经历 decode → 拼接对话模板 → re-tokenize 的往返路径。这一过程会引入额外的控制 token（如对话模板标记），使重新编码后的序列与原始序列产生偏差。

因果链：偏差后的 token 序列 → 条件向量改变 → 扩散模型据此生成的样本偏离训练端预期 → 奖励信号失去稳定锚点 → 策略更新方向紊乱 → 宏观表现为语义丢失或质量震荡。

修复机制：VeRL-Omni 在训练与 rollout 两端共享同一份 H3 原生 `input_ids`，彻底跳过 decode/re-tokenize 环节。核心约束：训练使用的文本条件必须与 rollout 完全相同。

排查清单与小结

三项故障分别发生在数学约定层（时间步方向）、工程接口层（权重布局）和数据流层（token 编解码），共同构成多模态扩散 RL 接入中的一组标准排查清单。共同教训是：在将任何新模型接入通用框架前，应逐一核验时间步语义、权重切分映射和 token 流一致性三项隐性契约，将运行时的静默故障前移为启动时的显式校验。

逻辑正确性问题解决后，工程瓶颈转移到物理硬件的显存限制上。

六、显存破局：大参数编码器的并行切分策略

一个编码器吃掉整张卡

MiniMax-H3 的文本编码器参数规模达到 32B，按 bf16 精度粗略估算仅静态权重即需约 64 GB 显存 - 已逼近单张 H100（80 GB）的物理上限，训练时还需叠加优化器状态、激活值和其他模型组件。

据演讲者在技术分享中的说明，在不开启编码器层面的并行时，仅第 0 号 GPU 的峰值显存占用就超过 100 GB。这是典型的显存墙（Memory Wall，单设备物理显存成为训练硬性天花板的现象）问题。

解法：ETP - 专为编码器设计的张量并行

张量并行（Tensor Parallelism, TP）是将单个算子沿特定维度切分到多张 GPU 上并行执行的技术，可同时降低单卡显存与分摊计算量。VeRL-Omni 在此基础上引入了 ETP（Encoder Tensor Parallelism，编码器张量并行） - 专门针对编码器独立开启的张量并行，与主干网络的 TP 度数可分别配置。

两者的关系：

维度	TP（主干张量并行）	ETP（编码器张量并行）
作用对象	视频生成主干网络（33B DiT）	文本编码器（32B）
切分目标	分摊主干计算与显存	化解编码器显存瓶颈
是否可独立设置	是	是

关键因果链：32B 编码器权重 → 单卡显存溢出（>100 GB） → 引入 ETP 切分编码器参数 → 单卡仅承载 1/ETP 的编码器权重 → 显存降至可用区间。当 ETP=4 时，每张卡仅需存放约 8B 参数对应的编码器权重份额。

实验锚点：FL2VA 训练配置

下图展示了 FL2VA 验证实验的具体配置与训练设置，是理解 ETP 如何在实际训练中发挥作用的数据锚点。

Blog experiment setup

The blog validates end-to-end training behavior rather than claiming a new benchmark record

Data

DanceGRPO ConsistID contains 27,815 English prompts.

FLUX.1-dev generates FL2VA condition images. Seed 42 splits 27,687 train and 128 test samples.

T2VA

8 GPUs with TP2.

Train at 256 x 384. Validate at 512 x 768 with 121 frames.

Rollout n=16, 10 sampling steps, and 40 validation steps.

FL2VA

8 GPUs with TP4 and ETP(encoder TP)=4.

Train at 288 x 448. Validate at 576 x 928. NUM_FRAMES 96 aligns to 107 frames.

LoRA rank 64 and alpha 128.

Reward combines CLAP and ImageBind AV. The old policy updates every two training steps.

图注：FL2VA 训练实验的并行策略与分辨率设置。来源：演讲 PPT，第 16 页

VeRL-Omni公开的
FL2VA（First/Last-frame-to-Video-Audio，首末帧引导视频音频生成）验证实验核心参数如下：

参数	配置值
GPU 数量	8
主干 TP	4
ETP（编码器 TP）	4
训练分辨率	288 × 448

需要明确：这组配置的目的是验证端到端训练行为的正确性，并非追求基准排名。288×448
的训练分辨率是显存预算约束下的工程折中。

ETP=4 的设置直接反映了 32B 编码器的额外显存压力。在编码器参数量较小的场景中，标准 TP 足以覆盖全模型显存需求，无需独立 ETP 维度；但当编码器膨胀至 32B 时，单一 TP 度数无法同时兼顾主干和编码器 - 盲目提高 TP 会增加主干通信开销，但不提高则编码器装不下。ETP 的工程价值在于将两者的并行度解耦，允许独立调节。

边界： 演讲材料未给出开启 ETP=4 后的精确单卡显存值。ETP 引入的跨卡 All-Reduce 通信对编码器前向延迟的量化影响同样未提供。若 GPU 显存进一步缩小或编码器继续增长，是否需要 ETP > 4 甚至叠加流水线并行，仍有待工程验证。显存约束化解之后，计算效率成为新的焦点，特别是小 Batch Size 下的访存受限问题。

七、性能跃升：连续批处理突破访存瓶颈

小 Batch Size 下计算单元为何大量闲置

在实际 rollout 阶段，一个现实矛盾随即暴露：当 batch size 为 1 时，GPU 的浮点运算单元几乎没有被喂饱，系统性能瓶颈落在显存读写速度上。

瓶颈类型	英文	含义	典型表现
计算受限	Compute bound	GPU 算力先被耗尽	增大 batch 会线性增加耗时
访存受限	Memory bound	显存带宽先被耗尽	计算单元空转等待数据搬运

判定标准是算术强度 (Arithmetic Intensity) - 每字节访存对应的浮点操作数。当算术强度低于硬件的"计算-带宽比"临界值时，系统即进入 Memory bound 状态。

扩散模型在去噪推理时，每一步需对整个 Transformer 权重做一次完整读取。当 batch size 仅为 1 时，权重搬运开销被单条样本独自承担，算术强度极低。据演讲者实测经验：视频生成模型（如 MiniMax-H3）在单样本时可能仍处于 Compute bound 状态（因单步计算量本身极大）；而图像生成模型由于单步计算量更轻，更容易落入 Memory bound 区间 - 计算单元远未打满，带宽已成为短板。

工程手段：请求级连续批处理

连续批处理 (Continuous Batching) 是在推理服务中已被广泛验证的调度策略：当某条请求完成或出现空隙时，新请求可立即填入空闲计算槽位，无需等待整 batch 结束。

vLLM-Omni 采用的粒度为请求级批处理 (Request-level Batching) - 将多条独立生成请求拆分后交错调度。核心优势：

- 1 实现简单：只需在调度层对请求做分桶，不涉及算子层改动。
- 2 回避分辨率不齐的问题：图像生成中不同请求的分辨率往往不同，难以直接拼成规整 batch；请求级调度天然规避了这一限制。
- 3 提升带宽利用率：多条请求交叠执行时，一条请求的计算阶段可与另一条的访存阶段重叠，有效隐藏搬运延迟。

收益量化与边界条件

据演讲者在技术问答环节的经验分享（口头陈述，非正式基准测试），在图像模型上开启 request-level continuous batching 后，rollout 阶段的生成耗时可减少约 50%。以下限定需注意：

- 硬件依赖：收益大小取决于具体 GPU 的算力与带宽之比，"计算能力很强但访存带宽相对弱"的设备获益最大。
- 模型规模：模型越小，单步算术强度越低，开启后的提升幅度越明显。
- 视频模型受限：MiniMax-H3 等视频生成模型单样本显存占用远大于图像模型，且若模型本身处于 Compute bound 则无带宽层面的收益。

- 演讲材料未给出不同 GPU 型号、不同分辨率下的详细对照数据。

小结：小 Batch Size 场景下的性能瓶颈本质上是算术强度不足导致的 Memory bound 问题。请求级连续批处理通过交错调度多条请求，在不大幅增加显存占用的前提下提高了带宽利用率。至此，从异步数据流、工程排雷、显存切分到吞吐优化的各环节均已就绪，最后需要将它们串联为一个完整的端到端训练闭环。

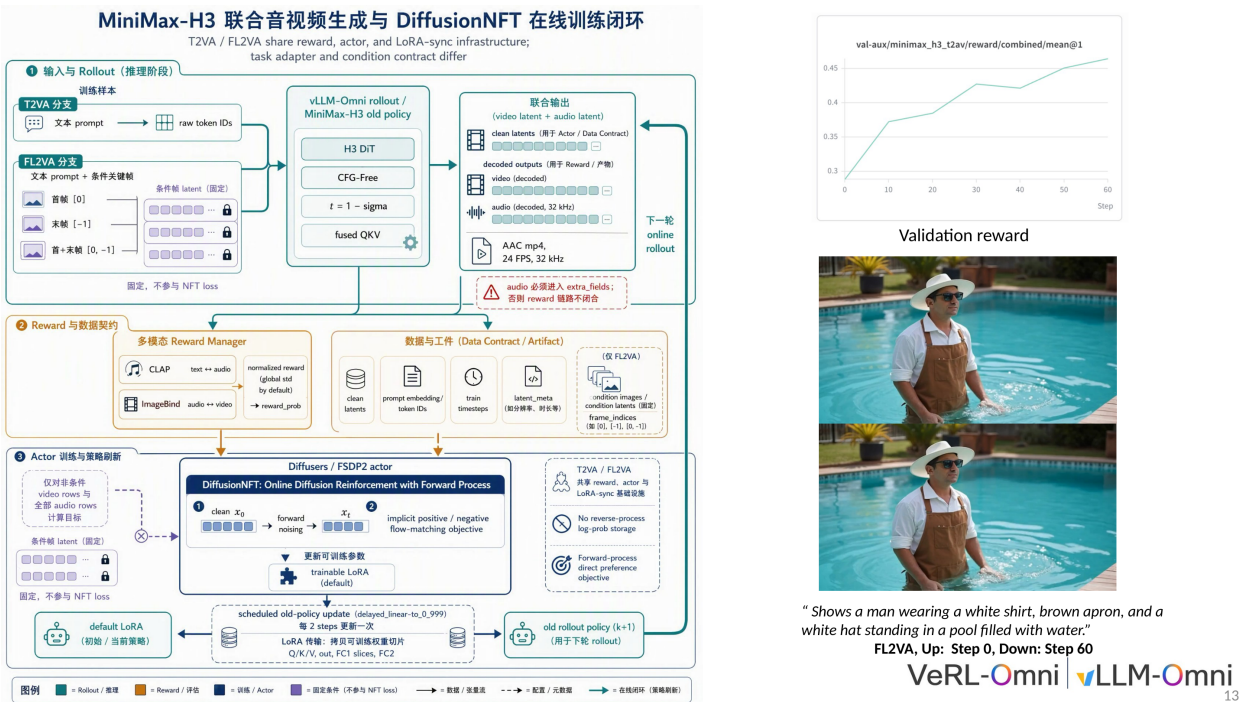
八、闭环与演进：音视频联合训练与未来架构

音视频联合生成的在线闭环训练

多模态 RL 的终极挑战不是单独优化某一模态，而是让视频帧与音频波形在同一训练循环中协同改进。这带来三个工程要求：1) 视听潜变量的生成频率不同（视频帧率与音频采样率存在数量级差距），需要统一的数据契约；2) 奖励信号必须同时评估跨模态一致性；3) 策略更新需要与高吞吐推理解耦。

端到端闭环全景

下图是 MiniMax-H3 完整训练闭环的总览，涵盖从推理生成到奖励计算再到策略更新的完整数据流，是理解各组件如何最终协同工作的全景图。



图注：MiniMax-H3 端到端在线训练闭环，涵盖 Rollout、Reward 和 Actor Update 三阶段。来源：演讲 PPT，第 13 页

闭环分为三个顺序阶段：

阶段一 · Rollout : vLLM-Omni 加载 H3 DiT 模型，并行生成视频潜变量与音频潜变量。支持两种任务模式 -

T2VA (Text-to-Video-Audio, 纯文本条件) 和 FL2VA (First/Last-frame-to-Video-Audio, 额外注入首末帧图像条件)。两种模式共享奖励模型、Actor 网络和 LoRA 同步基础设施, 仅在任务适配器和条件契约上有差异。

阶段二 · Reward: 奖励管理器同时调用两个跨模态评估模型:

- CLAP (Contrastive Language-Audio Pretraining, 对比语言-音频预训练模型): 衡量生成音频与文本描述之间的语义匹配度。
- ImageBind (多模态绑定模型): 衡量生成音频与视频帧之间的跨模态一致性。

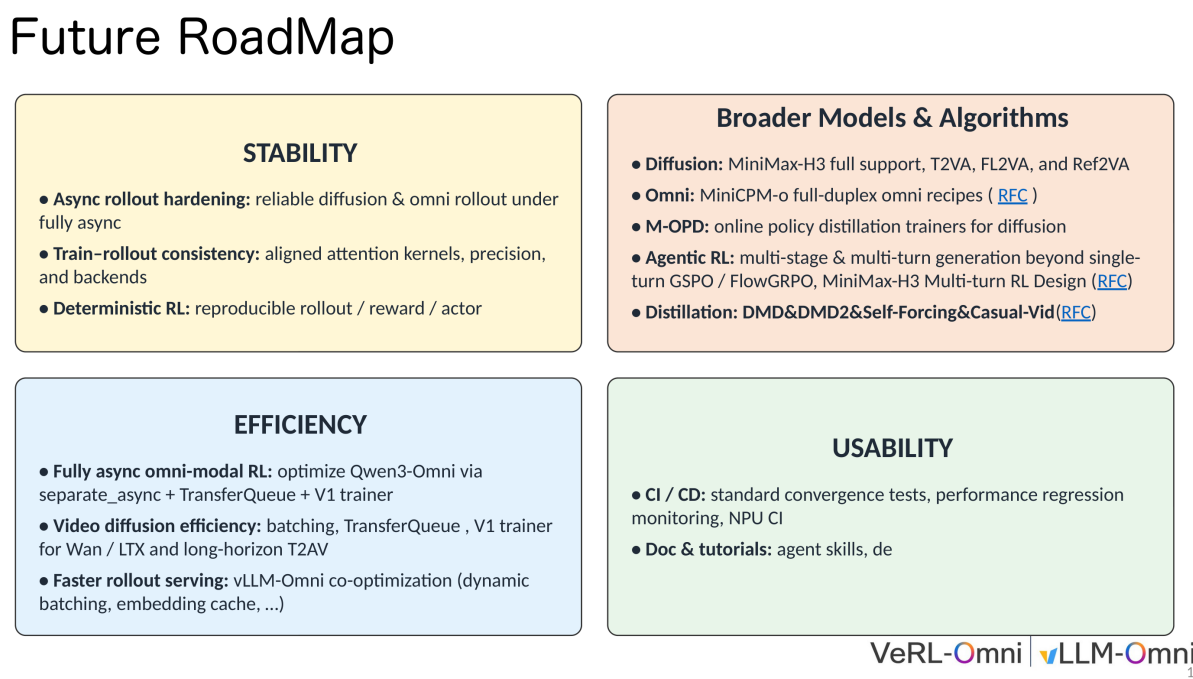
两个分数联合构成标量奖励信号。关键工程约束: 音频数据必须通过 `extra_fields` 字段进入数据契约, 否则 CLAP 和 ImageBind 都无法获得音频输入, 奖励链路将断裂。

阶段三 · Actor Update: Actor 网络使用 DiffusionNFT 进行参数更新。训练后的 LoRA 权重同步回 rollout 引擎的旧策略, 采用延迟更新机制 - 旧策略每 2 步更新一次, 而非每步即时同步, 以防止策略剧烈跳变导致 rollout 分布突变。

从图中右侧的验证奖励曲线可以看出, FL2VA 任务在训练过程中生成质量持续提升。

未来演进路线

下图按四个维度展示 VeRL-Omni 的后续发展方向, 每个象限解决一类系统瓶颈。



图注: VeRL-Omni 未来路线图, 按稳定性、算法广度、效率和易用性四个维度展开。来源: 演讲 PPT, 第 17 页

- 1 稳定性: 推进异步 rollout 加固、训练-推理一致性对齐 (注意力核、精度、后端三方面) 以及可确定性复现的 RL 流程。
- 2 算法广度: 从单轮生成扩展到 Agentic RL (一种支持多阶段和多轮生成的强化学习方法), 以应对多步骤复合任务; 同时引入在线策略蒸馏等方案。

- 3 效率：实现全异步多模态 RL，通过 TransferQueue + 新一代 trainer 架构消除 rollout 与训练之间的同步阻塞，并对 vLLM-Omni 进行动态批处理和嵌入缓存联合优化。
- 4 易用性：建立标准收敛测试和性能回归监控的 CI/CD 体系。

边界：若音频未正确写入 `extra_fields`，闭环在奖励阶段即中断。条件潜变量（如 FL2VA 的首末帧）被冻结，RL 无法修正其质量。演讲材料未给出 CLAP 与 ImageBind 在联合奖励中各自的权重配比。路线图中提到的 Agentic RL 和策略蒸馏目前处于社区 RFC 或探索阶段，尚未完全实现。

结论

核心发现：

- 1 生成范式差异是多模态 RL 的根本矛盾。自回归的可变长 token 序列与扩散模型的固定步数潜变量轨迹在采样方式、显存特征和梯度回传路径上全面不兼容。VeRL-Omni 通过替换推理后端（vLLM-Omni）并支持专用算法，从架构层面化解了这一冲突。
- 2 三引擎解耦 + 异步数据流是系统设计的关键抽象。将轨迹生成、奖励打分和策略优化映射为独立引擎，用 TransferQueue 实现流水线执行，在 Qwen-Image + FlowGRPO 条件下带来约 20% 的端到端吞吐提升。
- 3 DiffusionNFT 的标量奖励加权机制有效规避了成对数据构造的组合爆炸。通过隐式正负策略插值和 $r \in [0, 1]$ 的连续加权，偏好对齐在连续生成空间中得以低成本落地。
- 4 时间步对齐、LoRA 权重同步和 Token 漂移是多模态扩散 RL 集成的三大隐性故障源。它们分别发生在数学约定、工程接口和数据流三个层次，不会在编译阶段暴露，必须依靠系统化排查清单在启动前完成显式校验。
- 5 ETP（编码器张量并行）是应对大参数编码器显存瓶颈的针对性策略。在 MiniMax-H3 的 32B 文本编码器场景下，ETP 将编码器并行度从主干中独立出来，使得在 8 卡 TP=4 + ETP=4 配置下完成 FL2VA 端到端训练验证。
- 6 小 Batch Size 场景的访存受限问题可通过请求级连续批处理缓解。该策略在图像生成模型上的收益更为显著，据经验数据可减少约 50% 的 rollout 耗时（具体收益取决于 GPU 算力/带宽比和模型规模，来源为演讲者口头分享而非正式基准测试）。
- 7 端到端训练闭环依赖 CLAP 和 ImageBind 的联合跨模态奖励信号。旧策略每 2 步延迟更新的机制是维持在线 RL 训练稳定性的重要设计。

局限与开放问题：

- 20% 吞吐提升仅在 Qwen-Image + FlowGRPO 条件下验证，其他模态和算法的量化表现未公开。
- ETP 引入的跨卡通信延迟、开启 ETP 后的精确单卡显存值均无公开数据。
- 50% rollout 加速来自演讲者的经验分享，缺少 GPU 型号和分辨率的对照条件。
- Agentic RL、在线策略蒸馏等路线图条目仍处于社区探索阶段，尚未实际落地。

- 联合奖励中 CLAP 与 ImageBind 的权重配比以及 DiffusionNFT 中 β 的敏感性分析，演讲材料均未给出。